

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Knowledge Representation Design
Assessment

COURSE NAME: **ARTIFICIAL INTELLIGENCE COURSE CODE: MLA01**
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO3: *Implement propositional and first-order logic using resolution, unification, and inference techniques for knowledge representation and reasoning.(BTL 3)*

Assessment Tool Weightage: 40 %

Total Marks: 100

Time: 60 Mins

No. of Questions: 5

Annexure A
Knowledge Representation Design Assessment

Code of Conduct:

I K Sathwik Kumar (192425051) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K Sathwik Kumar

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Engineering and Knowledge Base Design	15%	
3. Propositional Logic Representation	10%	
4. First-Order Logic Representation	15%	
5. Unification and Resolution	15%	
6. Forward and Backward Chaining	15%	
7. Inference and Reasoning Accuracy	10%	

8. System Architecture / Representation Diagram	5%	
9. Prototype Implementation and Results	5%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE QUESTION

1. Smart Healthcare Diagnosis System

A hospital wants to develop an AI-based system that can identify possible illnesses from patient symptoms. The system contains facts such as:

- Ravi has fever.
- Ravi has cough.
- Ravi has body pain.
- Patients with fever and cough may have flu.
- Patients with flu and body pain may have viral infection.
- Patients with viral infection should be advised to rest.

Task:

1. Represent the given knowledge using **Propositional Logic**.
2. Convert the knowledge into **First-Order Logic (FOL)**.
3. Construct a knowledge base containing facts and rules.
4. Demonstrate **unification** for at least two predicates.
5. Use **forward chaining** to determine Ravi's condition.
6. Use **backward chaining** to prove whether Ravi needs rest.
7. Demonstrate **resolution** for one selected query.
8. Explain the final inference produced by the system.

2. Intelligent Loan Approval System

A bank wants to develop an AI system to determine whether a customer is eligible for a loan. The knowledge base contains information about income, credit score, employment, existing loans, and repayment history.

For example:

- Arun has a stable job.
- Arun has a high credit score.
- Arun has sufficient income.
- Customers with stable employment and sufficient income are eligible for preliminary approval.
- Customers with high credit scores and good repayment history are considered low-risk.
- Low-risk customers with preliminary approval can be recommended for loan approval.

Task:

1. Identify the **entities, facts, predicates, and rules** involved.
2. Represent the knowledge using **First-Order Logic**.
3. Build a knowledge base with at least **10 facts and 8 rules**.
4. Demonstrate **unification and substitution**.
5. Apply **forward chaining** to determine whether Arun can be recommended for approval.
6. Apply **backward chaining** for the query `LoanApproved(Arun)`.
7. Use **resolution** to prove or disprove one query.
8. Discuss how the knowledge-engineering process helps maintain the loan rules.

3. Intelligent Vehicle Fault Diagnosis

An automobile service center wants to develop an expert system that identifies possible vehicle faults based on symptoms.

The system knows that:

- A vehicle has a dead battery if it does not start and the headlights are dim.
- A vehicle may have a fuel problem if the engine cranks but does not start.
- A vehicle may have an overheating problem if the temperature is high and coolant is low.
- A vehicle should be sent for immediate service if overheating is detected.

For a vehicle named **Car101**, the system receives:

- Car101 does not start.
- Car101 has dim headlights.
- Car101 has a high temperature.

Task:

1. Design the **knowledge representation** for the vehicle diagnosis system.
2. Represent at least five rules using **FOL**.
3. Represent selected statements using **Propositional Logic**.
4. Demonstrate **unification** using suitable predicates.
5. Apply **forward chaining** to identify possible faults.
6. Apply **backward chaining** for the query `NeedsService(Car101)`.
7. Demonstrate **resolution** for one diagnosis.
8. Explain how inference is used to arrive at the final diagnosis.

4. Intelligent Student Academic Advising System

A university wants an AI-based academic advising system that recommends actions to students based on their academic performance.

The system contains rules such as:

- Students with attendance below 75% require attendance counseling.
- Students who fail two or more courses require academic counseling.
- Students with high attendance and good academic performance can be recommended for advanced courses.
- Students who have completed prerequisite courses can register for the corresponding advanced course.

Consider a student **Priya** who has:

- Attendance of 68%.
- Two failed courses.
- Completed the prerequisite for an advanced AI course.

Task:

1. Identify the **knowledge engineering requirements** for the system.
2. Represent the academic rules using **FOL**.
3. Create a knowledge base containing facts and rules.
4. Demonstrate **unification** using student and course predicates.
5. Use **forward chaining** to identify recommendations for Priya.
6. Use **backward chaining** to determine whether Priya requires academic counseling.
7. Apply **resolution** to prove one selected recommendation.
8. Explain the advantages and limitations of using a rule-based inference system for academic advising.

5. Cybersecurity Threat Detection System

An organization wants to develop an AI knowledge-based system to identify suspicious activities in its network.

The knowledge base contains rules such as:

- Multiple failed login attempts from the same user may indicate a brute-force attack.
- A login from an unusual location combined with multiple failed attempts indicates suspicious activity.
- Suspicious activity followed by unauthorized file access indicates a possible security breach.
- A possible security breach should generate a high-priority alert.

For user **User101**, the system observes:

- Five failed login attempts.
- Login from an unusual location.

- Unauthorized access to a confidential file.

Task:

1. Design a **knowledge representation model** for the cybersecurity domain.
2. Represent the knowledge using **Propositional Logic and FOL**.
3. Construct at least **10 facts and 8 rules**.
4. Demonstrate **unification and substitution**.
5. Apply **forward chaining** to determine whether a security alert should be generated.
6. Apply **backward chaining** for the query `HighPriorityAlert(User101)`.
7. Use **resolution** to prove the existence of a possible security breach.
8. Compare **forward chaining and backward chaining** for cybersecurity reasoning and justify which is more suitable.

1. Smart Healthcare Diagnosis System

1.1 Problem Analysis

The objective is to build a rule-based healthcare diagnosis system that identifies possible illnesses from symptoms.

Entities

- Ravi
- Patient
- Flu
- Viral infection
- Rest

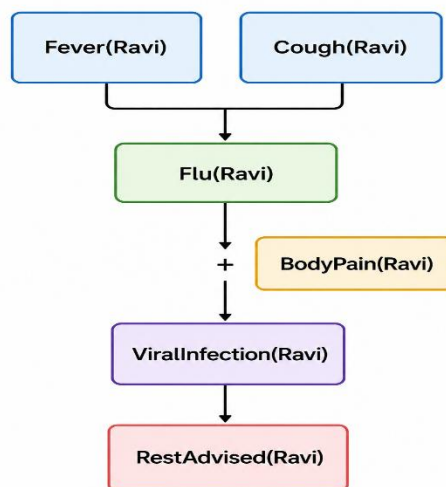
Initial facts

- Ravi has fever.
- Ravi has cough.
- Ravi has body pain.

Rules

1. Fever + cough $\rightarrow$ Flu
2. Flu + body pain $\rightarrow$ Viral infection
3. Viral infection $\rightarrow$ Rest is advised

The reasoning chain is:



1.2 Propositional Logic Representation

Let:

- F = Ravi has fever
- C = Ravi has cough
- B = Ravi has body pain
- L = Ravi has flu
- V = Ravi has viral infection
- R = Ravi should rest

Facts:

F

C

B

Rules:

$(F \wedge C) \rightarrow L$

$(L \wedge B) \rightarrow V$

$V \rightarrow R$

Therefore:

$F \wedge C \wedge B$

$(F \wedge C) \rightarrow L$

$(L \wedge B) \rightarrow V$

$V \rightarrow R$

Hence:

R

Therefore, Ravi should be advised to rest.

1.3 First-Order Logic Representation

Predicates:

Fever(x)

Cough(x)

BodyPain(x)

Flu(x)

VirallInfection(x)

RestAdvised(x)

Facts:

Fever(Ravi)

Cough(Ravi)

BodyPain(Ravi)

Rules:

$\forall x [Fever(x) \wedge Cough(x) \rightarrow Flu(x)]$

$\forall x [Flu(x) \wedge BodyPain(x) \rightarrow VirallInfection(x)]$

$\forall x [VirallInfection(x) \rightarrow RestAdvised(x)]$

1.4 Knowledge Base

Facts

Fever(Ravi)

Cough(Ravi)

BodyPain(Ravi)

Rules

R1: $Fever(x) \wedge Cough(x) \rightarrow Flu(x)$

R2: $Flu(x) \wedge BodyPain(x) \rightarrow VirallInfection(x)$

R3: $VirallInfection(x) \rightarrow RestAdvised(x)$

1.5 Unification

Example 1

Consider:

Fever(x)

Fever(Ravi)

Substitution:

$\{x/Ravi\}$

After substitution:

Fever(Ravi)

Therefore, the two predicates unify.

Example 2

Consider:

Flu(x)

Flu(Ravi)

Most General Unifier:

MGU = $\{x/Ravi\}$

After substitution:

Flu(Ravi)

1.6 Forward Chaining

Forward chaining starts from known facts and repeatedly applies rules.

Step 1

Initial facts:

Fever(Ravi)

Cough(Ravi)

BodyPain(Ravi)

Step 2

R1 requires:

$Fever(Ravi) \wedge Cough(Ravi)$

Both are available.

Therefore:

Flu(Ravi)

Step 3

R2 requires:

$Flu(Ravi) \wedge BodyPain(Ravi)$

Both are available.

Therefore:

ViralInfection(Ravi)

Step 4

R3 requires:

ViralInfection(Ravi)

Therefore:

RestAdvised(Ravi)

Forward-chaining result

Fever(Ravi)
+
Cough(Ravi)
↓
Flu(Ravi)
+
BodyPain(Ravi)
↓
ViralInfection(Ravi)
↓
RestAdvised(Ravi)

1.7 Backward Chaining

Query:

RestAdvised(Ravi)?

The system searches for a rule that concludes RestAdvised(x).

Rule:

$\text{ViralInfection}(x) \rightarrow \text{RestAdvised}(x)$

Therefore, it needs to prove:

$\text{ViralInfection}(\text{Ravi})$

Rule:

$\text{Flu}(x) \wedge \text{BodyPain}(x) \rightarrow \text{ViralInfection}(x)$

Therefore, it needs:

$\text{Flu}(\text{Ravi})$

$\text{BodyPain}(\text{Ravi})$

$\text{BodyPain}(\text{Ravi})$ is a fact.

To prove $\text{Flu}(\text{Ravi})$:

$\text{Fever}(x) \wedge \text{Cough}(x) \rightarrow \text{Flu}(x)$

Both:

$\text{Fever}(\text{Ravi})$

$\text{Cough}(\text{Ravi})$

are facts.

Therefore:

$\text{Flu}(\text{Ravi})$

Then:

$\text{ViralInfection}(\text{Ravi})$

and finally:

$\text{RestAdvised}(\text{Ravi})$

Thus, the query is proved.

1.8 Resolution

Selected query:

$\text{ViralInfection}(\text{Ravi})$

Convert the rule:

$\text{Flu}(x) \wedge \text{BodyPain}(x) \rightarrow \text{ViralInfection}(x)$

to clause form:

$\neg \text{Flu}(x) \vee \neg \text{BodyPain}(x) \vee \text{ViralInfection}(x)$

Facts:

$\text{Flu}(\text{Ravi})$

$\text{BodyPain}(\text{Ravi})$

Negate the query:

$\neg \text{ViralInfection}(\text{Ravi})$

Resolve:

$\neg \text{Flu}(\text{Ravi}) \vee \neg \text{BodyPain}(\text{Ravi}) \vee \text{ViralInfection}(\text{Ravi})$

with:

$\neg \text{ViralInfection}(\text{Ravi})$

gives:

$\neg \text{Flu}(\text{Ravi}) \vee \neg \text{BodyPain}(\text{Ravi})$

Resolve with:

$\text{Flu}(\text{Ravi})$

gives:

$\neg \text{BodyPain}(\text{Ravi})$

Resolve with:

$\text{BodyPain}(\text{Ravi})$

gives:

□

The empty clause means contradiction, so the original query is proved.

1.9 Final Inference

The system concludes:

Ravi has flu.

Ravi may have a viral infection.

Ravi should be advised to rest.

This is a **rule-based inference**, not a clinical diagnosis. In a real healthcare system, additional medical evidence and professional validation would be required.

2. Intelligent Loan Approval System

2.1 Problem Analysis

The objective is to determine whether a customer can be recommended for a loan based on employment, income, credit score, repayment history, and risk.

Entities

- Arun
- Other customers
- Loans
- Employment status
- Credit score
- Income
- Repayment history
- Risk level

Predicates

StableJob(x)

HighCreditScore(x)

SufficientIncome(x)

GoodRepayment(x)

ExistingLoan(x)

PreliminaryApproval(x)

LowRisk(x)

DocumentsVerified(x)

Eligible(x)

LoanReviewed(x)

LoanApproved(x)

Recommended(x)

2.2 First-Order Logic Representation

Facts:

StableJob(Arun)

HighCreditScore(Arun)

SufficientIncome(Arun)

GoodRepayment(Arun)

NoDefaultHistory(Arun)

DocumentsVerified(Arun)

IdentityVerified(Arun)

ExistingLoan(Arun)

PositiveBankHistory(Arun)

AgeEligible(Arun)

Rules:

R1: $\text{StableJob}(x) \wedge \text{SufficientIncome}(x) \rightarrow \text{PreliminaryApproval}(x)$

R2: $\text{HighCreditScore}(x) \wedge \text{GoodRepayment}(x) \rightarrow \text{LowRisk}(x)$

R3: $\text{LowRisk}(x) \wedge \text{PreliminaryApproval}(x) \rightarrow \text{Recommended}(x)$

R4: $\text{GoodRepayment}(x) \wedge \text{NoDefaultHistory}(x) \rightarrow \text{ReliableCustomer}(x)$

R5: $\text{IdentityVerified}(x) \wedge \text{DocumentsVerified}(x) \rightarrow \text{VerifiedCustomer}(x)$

R6: $\text{AgeEligible}(x) \wedge \text{VerifiedCustomer}(x) \rightarrow \text{Eligible}(x)$

R7: $\text{Eligible}(x) \wedge \text{ReliableCustomer}(x) \rightarrow \text{LoanReviewed}(x)$

R8: $\text{LoanReviewed}(x) \wedge \text{Recommended}(x) \rightarrow \text{LoanApproved}(x)$

2.3 Knowledge Base

10 Facts

F1 $\text{StableJob}(\text{Arun})$

F2 $\text{HighCreditScore}(\text{Arun})$

F3 $\text{SufficientIncome}(\text{Arun})$

F4 $\text{GoodRepayment}(\text{Arun})$

F5 $\text{NoDefaultHistory}(\text{Arun})$

F6 $\text{DocumentsVerified}(\text{Arun})$

F7 $\text{IdentityVerified}(\text{Arun})$

F8 $\text{ExistingLoan}(\text{Arun})$

F9 $\text{PositiveBankHistory}(\text{Arun})$

F10 $\text{AgeEligible}(\text{Arun})$

8 Rules

R1 $\text{StableJob}(x) \wedge \text{SufficientIncome}(x) \rightarrow \text{PreliminaryApproval}(x)$

R2 $\text{HighCreditScore}(x) \wedge \text{GoodRepayment}(x) \rightarrow \text{LowRisk}(x)$

R3 $\text{LowRisk}(x) \wedge \text{PreliminaryApproval}(x) \rightarrow \text{Recommended}(x)$

R4 $\text{GoodRepayment}(x) \wedge \text{NoDefaultHistory}(x) \rightarrow \text{ReliableCustomer}(x)$

R5 $\text{IdentityVerified}(x) \wedge \text{DocumentsVerified}(x) \rightarrow \text{VerifiedCustomer}(x)$

R6 $\text{AgeEligible}(x) \wedge \text{VerifiedCustomer}(x) \rightarrow \text{Eligible}(x)$

R7 $\text{Eligible}(x) \wedge \text{ReliableCustomer}(x) \rightarrow \text{LoanReviewed}(x)$

R8 $\text{LoanReviewed}(x) \wedge \text{Recommended}(x) \rightarrow \text{LoanApproved}(x)$

2.4 Unification and Substitution

Rule:

$\text{StableJob}(x) \wedge \text{SufficientIncome}(x) \rightarrow \text{PreliminaryApproval}(x)$

Fact:

$\text{StableJob}(\text{Arun})$

Unification gives:

$\{x/\text{Arun}\}$

Substitution into the rule:

$\text{StableJob}(\text{Arun}) \wedge \text{SufficientIncome}(\text{Arun})$

$\rightarrow \text{PreliminaryApproval}(\text{Arun})$

Another example:

$\text{HighCreditScore}(x)$

$\text{HighCreditScore}(\text{Arun})$

MGU:

{x/Arun}

2.5 Forward Chaining

Initial facts include:

StableJob(Arun)

SufficientIncome(Arun)

HighCreditScore(Arun)

GoodRepayment(Arun)

NoDefaultHistory(Arun)

IdentityVerified(Arun)

DocumentsVerified(Arun)

AgeEligible(Arun)

Step 1

R1:

$\text{StableJob}(\text{Arun}) \wedge \text{SufficientIncome}(\text{Arun})$

Therefore:

$\text{PreliminaryApproval}(\text{Arun})$

Step 2

R2:

$\text{HighCreditScore}(\text{Arun}) \wedge \text{GoodRepayment}(\text{Arun})$

Therefore:

$\text{LowRisk}(\text{Arun})$

Step 3

R3:

$\text{LowRisk}(\text{Arun}) \wedge \text{PreliminaryApproval}(\text{Arun})$

Therefore:

$\text{Recommended}(\text{Arun})$

Step 4

R4:

$\text{GoodRepayment}(\text{Arun}) \wedge \text{NoDefaultHistory}(\text{Arun})$

Therefore:

$\text{ReliableCustomer}(\text{Arun})$

Step 5

R5:

$\text{IdentityVerified}(\text{Arun}) \wedge \text{DocumentsVerified}(\text{Arun})$

Therefore:

$\text{VerifiedCustomer}(\text{Arun})$

Step 6

R6:

$\text{AgeEligible}(\text{Arun}) \wedge \text{VerifiedCustomer}(\text{Arun})$

Therefore:

$\text{Eligible}(\text{Arun})$

Step 7

R7:

$\text{Eligible}(\text{Arun}) \wedge \text{ReliableCustomer}(\text{Arun})$

Therefore:

$\text{LoanReviewed}(\text{Arun})$

Step 8

R8:

$\text{LoanReviewed}(\text{Arun}) \wedge \text{Recommended}(\text{Arun})$

Therefore:

$\text{LoanApproved}(\text{Arun})$

Final forward-chaining result

$\text{Recommended}(\text{Arun})$

$\text{LoanApproved}(\text{Arun})$

2.6 Backward Chaining

Query:

$\text{LoanApproved}(\text{Arun})?$

Find a rule concluding $\text{LoanApproved}(x)$:

$\text{LoanReviewed}(x) \wedge \text{Recommended}(x) \rightarrow \text{LoanApproved}(x)$

Substitution:

$\{x/\text{Arun}\}$

Subgoals:

$\text{LoanReviewed}(\text{Arun})$

$\text{Recommended}(\text{Arun})$

To prove $\text{Recommended}(\text{Arun})$:

$\text{LowRisk}(\text{Arun})$

$\text{PreliminaryApproval}(\text{Arun})$

To prove $\text{LowRisk}(\text{Arun})$:

$\text{HighCreditScore}(\text{Arun})$

$\text{GoodRepayment}(\text{Arun})$

Both are facts.

To prove $\text{PreliminaryApproval}(\text{Arun})$:

$\text{StableJob}(\text{Arun})$

$\text{SufficientIncome}(\text{Arun})$

Both are facts.

Thus:

$\text{Recommended}(\text{Arun})$

For $\text{LoanReviewed}(\text{Arun})$:

$\text{Eligible}(\text{Arun})$

$\text{ReliableCustomer}(\text{Arun})$

These are derived from the available facts.

Therefore:

$\text{LoanApproved}(\text{Arun})$

is proved.

2.7 Resolution

Query:

LoanApproved(Arun)

Clause from R8:

$\neg \text{LoanReviewed}(x) \vee \neg \text{Recommended}(x) \vee \text{LoanApproved}(x)$

Substitute $x = \text{Arun}$:

$\neg \text{LoanReviewed}(\text{Arun}) \vee \neg \text{Recommended}(\text{Arun}) \vee \text{LoanApproved}(\text{Arun})$

Negate query:

$\neg \text{LoanApproved}(\text{Arun})$

Resolution gives:

$\neg \text{LoanReviewed}(\text{Arun}) \vee \neg \text{Recommended}(\text{Arun})$

We have:

LoanReviewed(Arun)

Recommended(Arun)

Resolving both produces:

□

Therefore:

LoanApproved(Arun)

is proved.

2.8 Knowledge Engineering Process

Knowledge engineering helps maintain loan rules by:

1. Identifying domain experts and business requirements.
2. Converting banking policies into formal rules.
3. Separating facts from decision rules.
4. Updating rules when banking policies change.
5. Avoiding duplicate or contradictory rules.
6. Testing rules using sample customers.
7. Maintaining traceability from a decision to its supporting facts.
8. Making the system easier to audit and explain.

For example, if the bank changes its minimum credit-score requirement, only the relevant rule needs to be updated.

3. Intelligent Vehicle Fault Diagnosis

3.1 Knowledge Representation

Entities

Car101

Other vehicles

Battery

Fuel system

Coolant system

Engine

Service center

Predicates

DoesNotStart(x)

DimHeadlights(x)

EngineCrank(x)

FuelProblem(x)

HighTemperature(x)

LowCoolant(x)

Overheating(x)

DeadBattery(x)

NeedsService(x)

ImmediateService(x)

3.2 FOL Rules

At least five rules are required.

R1: $\text{DoesNotStart}(x) \wedge \text{DimHeadlights}(x) \rightarrow \text{DeadBattery}(x)$

R2: $\text{EngineCrank}(x) \wedge \text{DoesNotStart}(x) \rightarrow \text{FuelProblem}(x)$

R3: $\text{HighTemperature}(x) \wedge \text{LowCoolant}(x) \rightarrow \text{Overheating}(x)$

R4: $\text{Overheating}(x) \rightarrow \text{ImmediateService}(x)$

R5: $\text{ImmediateService}(x) \rightarrow \text{NeedsService}(x)$

R6: $\text{DeadBattery}(x) \rightarrow \text{NeedsBatteryInspection}(x)$

3.3 Facts

For Car101:

$\text{DoesNotStart}(\text{Car101})$

$\text{DimHeadlights}(\text{Car101})$

$\text{HighTemperature}(\text{Car101})$

The system does not have:

$\text{LowCoolant}(\text{Car101})$

Therefore, overheating cannot yet be logically established.

3.4 Propositional Logic

Let:

- D = Car101 does not start
- H = Car101 has dim headlights
- T = Car101 has high temperature
- C = Car101 has low coolant

- $B = \text{Car101}$ has a dead battery
- $O = \text{Car101}$ is overheating
- $S = \text{Car101}$ needs service

Facts:

D

H

T

Rules:

$D \wedge H \rightarrow B$

$T \wedge C \rightarrow O$

$O \rightarrow S$

Since C is not known, O cannot be inferred.

However:

$D \wedge H \rightarrow B$

can be inferred.

3.5 Unification

Rule:

$\text{DoesNotStart}(x) \wedge \text{DimHeadlights}(x) \rightarrow \text{DeadBattery}(x)$

Facts:

$\text{DoesNotStart}(\text{Car101})$

$\text{DimHeadlights}(\text{Car101})$

Unification:

$\{x/\text{Car101}\}$

Substitution gives:

$\text{DoesNotStart}(\text{Car101}) \wedge \text{DimHeadlights}(\text{Car101})$

$\rightarrow \text{DeadBattery}(\text{Car101})$

Therefore:

$\text{DeadBattery}(\text{Car101})$

3.6 Forward Chaining

Initial facts:

$\text{DoesNotStart}(\text{Car101})$

$\text{DimHeadlights}(\text{Car101})$

$\text{HighTemperature}(\text{Car101})$

Apply R1:

$\text{DoesNotStart}(\text{Car101}) \wedge \text{DimHeadlights}(\text{Car101})$

Therefore:

$\text{DeadBattery}(\text{Car101})$

R2 requires:

$\text{EngineCranks}(\text{Car101})$

which is unavailable.

R3 requires:

$\text{HighTemperature}(\text{Car101})$

LowCoolant(Car101)

Only high temperature is known.

Therefore:

Overheating(Car101)

cannot be inferred.

Final diagnosis:

DeadBattery(Car101)

is supported by the available evidence.

3.7 Backward Chaining

Query:

NeedsService(Car101)?

Search for a rule concluding NeedsService(x):

ImmediateService(x) $\rightarrow$ NeedsService(x)

Therefore the system must prove:

ImmediateService(Car101)

Rule:

Overheating(x) $\rightarrow$ ImmediateService(x)

Therefore it must prove:

Overheating(Car101)

Rule:

HighTemperature(x) $\wedge$ LowCoolant(x) $\rightarrow$ Overheating(x)

We have:

HighTemperature(Car101)

but we do not have:

LowCoolant(Car101)

Therefore:

NeedsService(Car101)

cannot be proved from the given knowledge base.

Important inference

The system should **not assume low coolant simply because the temperature is high.**

Thus:

NeedsService(Car101) = Not provable

Additional diagnostic information, such as coolant level, is required.

3.8 Resolution

Selected query:

DeadBattery(Car101)

Rule R1 in clause form:

$\neg \text{DoesNotStart}(x) \vee \neg \text{DimHeadlights}(x) \vee \text{DeadBattery}(x)$

Substitute:

$\neg \text{DoesNotStart}(\text{Car101}) \vee \neg \text{DimHeadlights}(\text{Car101})$

$\vee \text{DeadBattery}(\text{Car101})$

Negated query:

$\neg \text{DeadBattery}(\text{Car101})$

Resolve with the rule:

$\neg \text{DoesNotStart}(\text{Car101}) \vee \neg \text{DimHeadlights}(\text{Car101})$

Given:

$\text{DoesNotStart}(\text{Car101})$

$\text{DimHeadlights}(\text{Car101})$

successive resolution produces:

□

Therefore:

$\text{DeadBattery}(\text{Car101})$

is proved.

3.9 Final Diagnosis

The strongest supported diagnosis is:

Dead battery → Yes

Fuel problem → Cannot be established

Overheating → Cannot be established

Immediate service → Cannot be established

This demonstrates an important property of rule-based systems: **they infer only what is supported by the knowledge base.**

4. Intelligent Student Academic Advising System

4.1 Problem Analysis

The system recommends academic actions based on:

- Attendance
- Number of failed courses
- Academic performance
- Prerequisite completion
- Course eligibility

Student

Priya

Course

AdvancedAI

4.2 Knowledge Engineering Requirements

The system requires:

1. Student information.
2. Attendance records.
3. Course performance records.
4. Failed-course information.
5. Prerequisite information.
6. Course registration rules.
7. Academic counseling rules.
8. Attendance counseling rules.
9. Advanced-course eligibility rules.
10. A rule-based inference engine.
11. A knowledge base.
12. A recommendation/output module.

4.3 FOL Representation

Predicates:

AttendanceBelow75(x)

FailedTwoOrMore(x)

HighAttendance(x)

GoodPerformance(x)

CompletedPrerequisite(x,c)

RequiresAttendanceCounseling(x)

RequiresAcademicCounseling(x)

RecommendedAdvancedCourse(x,c)

CanRegister(x,c)

Rules:

R1: AttendanceBelow75(x) $\rightarrow$ RequiresAttendanceCounseling(x)

R2: FailedTwoOrMore(x) $\rightarrow$ RequiresAcademicCounseling(x)

R3: HighAttendance(x) $\wedge$ GoodPerformance(x)

$\rightarrow$ RecommendedAdvancedCourse(x,c)

R4: CompletedPrerequisite(x,c)

→ CanRegister(x,c)

R5: GoodPerformance(x) $\wedge$ HighAttendance(x)

→ AcademicGoodStanding(x)

R6: AcademicGoodStanding(x) $\wedge$ CompletedPrerequisite(x,c)

→ AdvancedCourseEligible(x,c)

4.4 Knowledge Base

Facts about Priya

AttendanceBelow75(Priya)

FailedTwoOrMore(Priya)

CompletedPrerequisite(Priya,AdvancedAI)

Additional academic facts can be represented as:

Attendance(Priya,68)

FailedCourses(Priya,2)

CompletedPrerequisite(Priya,AdvancedAI)

The numeric facts can be converted into rule-compatible predicates:

AttendanceBelow75(Priya)

FailedTwoOrMore(Priya)

4.5 Unification

Consider:

CompletedPrerequisite(x,c)

CompletedPrerequisite(Priya,AdvancedAI)

The MGU is:

{x/Priya, c/AdvancedAI}

Substitution produces:

CompletedPrerequisite(Priya,AdvancedAI)

Therefore, the predicates unify.

4.6 Forward Chaining

Initial facts:

AttendanceBelow75(Priya)

FailedTwoOrMore(Priya)

CompletedPrerequisite(Priya,AdvancedAI)

Step 1

R1:

AttendanceBelow75(Priya)

Therefore:

RequiresAttendanceCounseling(Priya)

Step 2

R2:

FailedTwoOrMore(Priya)

Therefore:

RequiresAcademicCounseling(Priya)

Step 3

R4:

CompletedPrerequisite(Priya,AdvancedAI)

Therefore:

CanRegister(Priya,AdvancedAI)

Final recommendations

RequiresAttendanceCounseling(Priya)

RequiresAcademicCounseling(Priya)

CanRegister(Priya,AdvancedAI)

Therefore, Priya should receive attendance counseling and academic counseling, and she can register for the Advanced AI course based on the stated prerequisite rule.

4.7 Backward Chaining

Query:

RequiresAcademicCounseling(Priya)?

Find rule:

FailedTwoOrMore(x) $\rightarrow$ RequiresAcademicCounseling(x)

Substitution:

{x/Priya}

Required subgoal:

FailedTwoOrMore(Priya)

This is an existing fact.

Therefore:

RequiresAcademicCounseling(Priya)

is proved.

4.8 Resolution

Selected query:

RequiresAttendanceCounseling(Priya)

Rule:

AttendanceBelow75(x) $\rightarrow$ RequiresAttendanceCounseling(x)

Clause form:

$\neg$ AttendanceBelow75(x) $\vee$ RequiresAttendanceCounseling(x)

Substitute:

$\neg$ AttendanceBelow75(Priya)

$\vee$ RequiresAttendanceCounseling(Priya)

Negate the query:

$\neg$ RequiresAttendanceCounseling(Priya)

Resolve:

$\neg$ AttendanceBelow75(Priya)

Given fact:

AttendanceBelow75(Priya)

Resolution gives:

□

Therefore:

RequiresAttendanceCounseling(Priya)

is proved.

4.9 Advantages and Limitations

Advantages

1. Easy to understand.
2. Rules are transparent.
3. Recommendations can be explained.
4. University policies can be represented directly.
5. New rules can be added.
6. Inference is consistent when the knowledge base is correct.

Limitations

1. It depends heavily on the quality of the rules.
2. It may not handle unusual student situations well.
3. Numeric academic information needs careful representation.
4. Conflicting rules can produce inconsistent recommendations.
5. It does not automatically learn new policies.
6. Human academic judgment may still be required.

5. Cybersecurity Threat Detection System

5.1 Problem Analysis

The system identifies suspicious network behavior and generates security alerts.

Entity

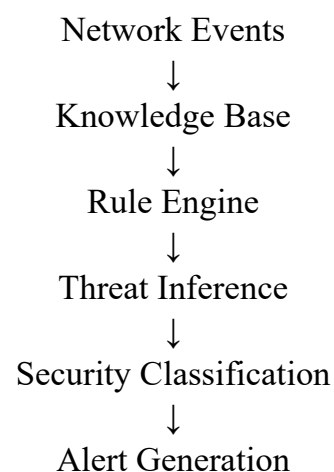
User101

Important concepts

- Failed logins
- Unusual location
- Suspicious activity
- Unauthorized file access
- Security breach
- High-priority alert
- Brute-force attack

5.2 Knowledge Representation Model

The system uses:



5.3 Propositional Logic

Let:

F = Multiple failed logins

U = Unusual login location

S = Suspicious activity

A = Unauthorized file access

B = Possible security breach

H = High-priority alert

Rules:

$F \rightarrow \text{BruteForceAttack}$

$U \wedge F \rightarrow S$

$S \wedge A \rightarrow B$

$B \rightarrow H$

For User101:

F = True

U = True

A = True

Therefore:

$F \rightarrow \text{BruteForceAttack}$

$U \wedge F \rightarrow \text{SuspiciousActivity}$

$\text{SuspiciousActivity} \wedge A \rightarrow \text{SecurityBreach}$

$\text{SecurityBreach} \rightarrow \text{HighPriorityAlert}$

5.4 First-Order Logic

Predicates:

MultipleFailedLogins(x)

UnusualLocation(x)

SuspiciousActivity(x)

UnauthorizedFileAccess(x)

BruteForceAttack(x)

PossibleSecurityBreach(x)

HighPriorityAlert(x)

KnownUser(x)

ConfidentialFileAccess(x)

SecurityIncident(x)

Rules:

R1: $\text{MultipleFailedLogins}(x) \rightarrow \text{BruteForceAttack}(x)$

R2: $\text{MultipleFailedLogins}(x) \wedge \text{UnusualLocation}(x) \rightarrow \text{SuspiciousActivity}(x)$

R3: $\text{SuspiciousActivity}(x) \wedge \text{UnauthorizedFileAccess}(x) \rightarrow \text{PossibleSecurityBreach}(x)$

R4: $\text{PossibleSecurityBreach}(x) \rightarrow \text{HighPriorityAlert}(x)$

R5: $\text{BruteForceAttack}(x) \rightarrow \text{SecurityIncident}(x)$
 R6: $\text{UnusualLocation}(x) \wedge \text{KnownUser}(x)$
 $\rightarrow \text{LocationAnomaly}(x)$
 R7: $\text{LocationAnomaly}(x) \wedge \text{MultipleFailedLogins}(x)$
 $\rightarrow \text{ElevatedThreat}(x)$
 R8: $\text{ElevatedThreat}(x) \wedge \text{UnauthorizedFileAccess}(x)$
 $\rightarrow \text{CriticalIncident}(x)$

5.5 Knowledge Base – 10 Facts

For User101:

F1 $\text{MultipleFailedLogins}(\text{User101})$
 F2 $\text{UnusualLocation}(\text{User101})$
 F3 $\text{UnauthorizedFileAccess}(\text{User101})$
 F4 $\text{KnownUser}(\text{User101})$
 F5 $\text{ConfidentialFileAccess}(\text{User101})$
 F6 $\text{LoginObserved}(\text{User101})$
 F7 $\text{NetworkActivityObserved}(\text{User101})$
 F8 $\text{AccountActive}(\text{User101})$
 F9 $\text{SecurityMonitoringEnabled}(\text{User101})$
 F10 $\text{AuthenticationServiceActive}(\text{User101})$

8 Rules

R1 $\text{MultipleFailedLogins}(x) \rightarrow \text{BruteForceAttack}(x)$
 R2 $\text{MultipleFailedLogins}(x) \wedge \text{UnusualLocation}(x)$
 $\rightarrow \text{SuspiciousActivity}(x)$
 R3 $\text{SuspiciousActivity}(x) \wedge \text{UnauthorizedFileAccess}(x)$
 $\rightarrow \text{PossibleSecurityBreach}(x)$
 R4 $\text{PossibleSecurityBreach}(x) \rightarrow \text{HighPriorityAlert}(x)$
 R5 $\text{BruteForceAttack}(x) \rightarrow \text{SecurityIncident}(x)$
 R6 $\text{UnusualLocation}(x) \wedge \text{KnownUser}(x)$
 $\rightarrow \text{LocationAnomaly}(x)$
 R7 $\text{LocationAnomaly}(x) \wedge \text{MultipleFailedLogins}(x)$
 $\rightarrow \text{ElevatedThreat}(x)$
 R8 $\text{ElevatedThreat}(x) \wedge \text{UnauthorizedFileAccess}(x)$
 $\rightarrow \text{CriticalIncident}(x)$

5.6 Unification and Substitution

Consider:

$\text{MultipleFailedLogins}(x)$

$\text{MultipleFailedLogins}(\text{User101})$

MGU:

$\{x/\text{User101}\}$

After substitution:

$\text{MultipleFailedLogins}(\text{User101})$

Another example:

UnauthorizedFileAccess(x)
UnauthorizedFileAccess(User101)

MGU:

{x/User101}

5.7 Forward Chaining

Initial facts:

MultipleFailedLogins(User101)
UnusualLocation(User101)
UnauthorizedFileAccess(User101)
KnownUser(User101)

Step 1

R1:

MultipleFailedLogins(User101)

Therefore:

BruteForceAttack(User101)

Step 2

R2:

MultipleFailedLogins(User101)

$\wedge$ UnusualLocation(User101)

Therefore:

SuspiciousActivity(User101)

Step 3

R3:

SuspiciousActivity(User101)

$\wedge$ UnauthorizedFileAccess(User101)

Therefore:

PossibleSecurityBreach(User101)

Step 4

R4:

PossibleSecurityBreach(User101)

Therefore:

HighPriorityAlert(User101)

Step 5

R6:

UnusualLocation(User101)

$\wedge$ KnownUser(User101)

Therefore:

LocationAnomaly(User101)

Step 6

R7:

LocationAnomaly(User101)

$\wedge$ MultipleFailedLogins(User101)

Therefore:

ElevatedThreat(User101)

Step 7

R8:

ElevatedThreat(User101)

$\wedge$ UnauthorizedFileAccess(User101)

Therefore:

CriticalIncident(User101)

Final result

BruteForceAttack(User101)

SuspiciousActivity(User101)

PossibleSecurityBreach(User101)

HighPriorityAlert(User101)

ElevatedThreat(User101)

CriticalIncident(User101)

5.8 Backward Chaining

Query:

HighPriorityAlert(User101)?

Find rule:

PossibleSecurityBreach(x)

$\rightarrow$ HighPriorityAlert(x)

Substitution:

{x/User101}

New goal:

PossibleSecurityBreach(User101)

Find rule:

SuspiciousActivity(x)

$\wedge$ UnauthorizedFileAccess(x)

$\rightarrow$ PossibleSecurityBreach(x)

New goals:

SuspiciousActivity(User101)

UnauthorizedFileAccess(User101)

UnauthorizedFileAccess(User101) is a fact.

To prove SuspiciousActivity(User101):

MultipleFailedLogins(x)

$\wedge$ UnusualLocation(x)

$\rightarrow$ SuspiciousActivity(x)

Both facts are available:

MultipleFailedLogins(User101)

UnusualLocation(User101)

Therefore:

SuspiciousActivity(User101)

Then:

PossibleSecurityBreach(User101)

and finally:

HighPriorityAlert(User101)

The query is proved.

5.9 Resolution

Selected query:

PossibleSecurityBreach(User101)

Rule R3:

SuspiciousActivity(x)

$\wedge$ UnauthorizedFileAccess(x)

$\rightarrow$ PossibleSecurityBreach(x)

Clause form:

$\neg$ SuspiciousActivity(x)

$\vee \neg$ UnauthorizedFileAccess(x)

$\vee$ PossibleSecurityBreach(x)

Substitute:

$\neg$ SuspiciousActivity(User101)

$\vee \neg$ UnauthorizedFileAccess(User101)

$\vee$ PossibleSecurityBreach(User101)

Negate the query:

$\neg$ PossibleSecurityBreach(User101)

Resolve:

$\neg$ SuspiciousActivity(User101)

$\vee \neg$ UnauthorizedFileAccess(User101)

Known facts:

SuspiciousActivity(User101)

UnauthorizedFileAccess(User101)

Resolve with the first fact:

$\neg$ UnauthorizedFileAccess(User101)

Resolve with the second fact:

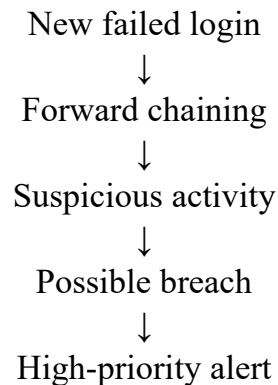
5.10 Forward Chaining vs Backward Chaining

Feature	Forward Chaining	Backward Chaining
Starting point	Known facts	Goal/query
Direction	Facts $\rightarrow$ conclusions	Goal $\rightarrow$ required facts
Best for	Continuous monitoring	Specific investigation
Cybersecurity use	Excellent for real-time event processing	Excellent for investigating a particular alert
Example	New failed login triggers rules	Check whether User101 is a threat
Potential issue	May generate many unnecessary conclusions	May miss useful conclusions outside the query

Which is more suitable?

For a real-time cybersecurity monitoring system, **forward chaining is generally more suitable** because network events continuously arrive and the system needs to react immediately.

For example:



However, backward chaining is useful for investigating a specific question such as: "Why did User101 receive a high-priority alert?"

Therefore, a practical cybersecurity system can use **both approaches**:

Forward chaining → Real-time detection

Backward chaining → Investigation and explanation

7. Overall Comparison of the Five Systems

System	Initial Evidence	Main Inference	Final Output
Healthcare	Fever, cough, body pain	Flu → viral infection	Rest advised
Loan	Job, income, credit score, repayment	Low risk → approval	Loan approved
Vehicle	No start, dim headlights	Dead battery	Battery diagnosis
Academic	Low attendance, failed courses, prerequisite	Counseling + registration	Academic recommendations
Cybersecurity	Failed logins, unusual location, unauthorized access	Suspicious activity → breach	High-priority alert

8. Overall Inference Engine Process

The five systems follow the same general reasoning cycle:

Step 1 – Collect Facts

The system receives facts from the real-world environment.

Step 2 – Store Facts

Facts are stored in the knowledge base.

Step 3 – Match Rules

The inference engine searches for rules whose conditions match existing facts.

Step 4 – Unification

Variables in rules are matched with constants in facts.

Example:

Fever(x)
Fever(Ravi)
gives:
{x/Ravi}

Step 5 – Forward Chaining

Known facts are used to derive new facts.

Step 6 – Backward Chaining

A desired conclusion is converted into smaller subgoals.

Step 7 – Resolution

The system can prove a query by assuming its negation and deriving a contradiction.

Step 8 – Final Decision

The derived conclusion is presented as the system's recommendation, diagnosis, approval, or alert.

9. Critical Analysis

A rule-based knowledge system provides several important benefits:

1. **Explainability:** Every conclusion can be traced to facts and rules.
2. **Consistency:** The same rules produce the same conclusion for the same evidence.
3. **Modularity:** Rules can be added, removed, or modified independently.
4. **Transparency:** Domain experts can inspect the knowledge base.
5. **Logical reasoning:** FOL supports variables, constants, relationships, and quantification.
6. **Efficient inference:** Forward and backward chaining allow systematic reasoning.
7. **Resolution:** Provides a formal mechanism for proving logical consequences.

However, the systems also have limitations:

- They depend on complete and accurate knowledge.
- They cannot automatically handle unknown situations unless rules are provided.
- Conflicting rules may create inconsistent conclusions.
- Uncertainty is difficult to represent using basic Boolean/FOL rules.
- Real-world domains often require probabilities, machine learning, fuzzy logic, or human judgment.

10. Final Conclusion

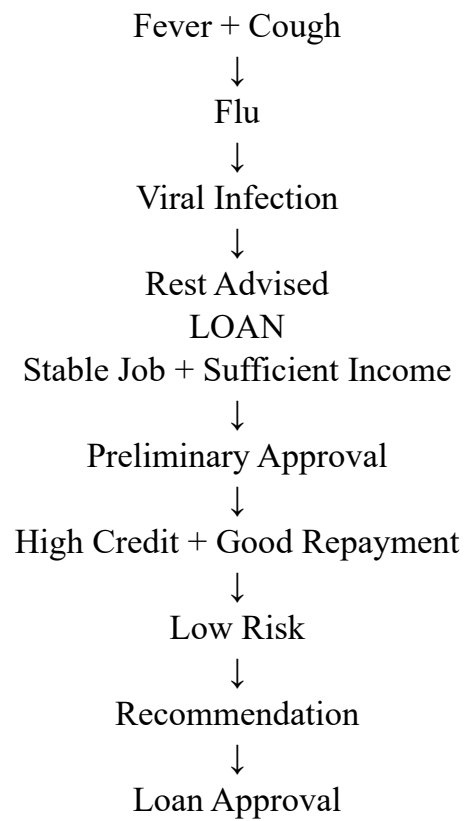
Knowledge representation and inference techniques provide a structured way to develop intelligent decision-support systems.

In the healthcare system, symptoms lead to possible conditions and recommendations. In the loan system, financial facts lead to risk assessment and approval recommendations. In the vehicle system, symptoms lead to fault diagnosis. In academic advising, student records lead to counseling and course recommendations. In cybersecurity, network events lead to threat detection and security alerts.

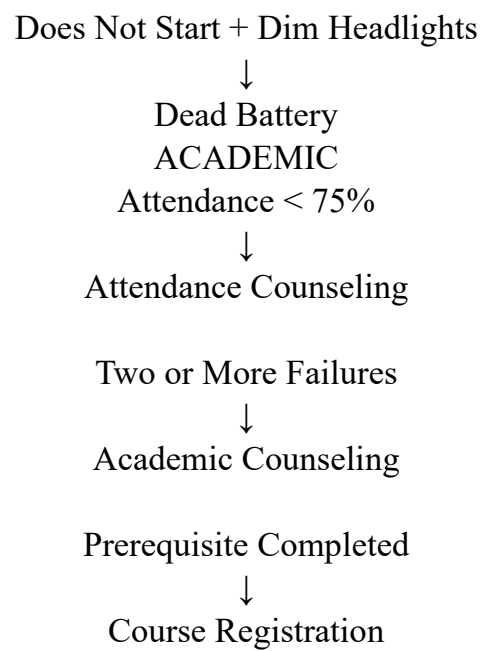
The combination of **Propositional Logic, First-Order Logic, knowledge bases, unification, substitution, forward chaining, backward chaining, and resolution** enables the systems to transform raw facts into logically supported conclusions.

The most important inference chains are:

HEALTHCARE



VEHICLE



CYBERSECURITY

Multiple Failed Logins + Unusual Location



Suspicious Activity



Unauthorized File Access



Possible Security Breach



High-Priority Alert

Evaluation Rubrics:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
Problem Analysis and Understanding	15%	13–15% – Clearly understands and analyzes the real-world problem, objectives, entities, facts, constraints, and expected inference.	10–12% – Understands the problem with minor omissions.	7–9% – Shows partial understanding with some important elements missing.	0–6% – Poor or incorrect understanding of the problem.
Knowledge Engineering & Knowledge Base Design	15%	13–15% – Develops a complete, consistent, and well-structured knowledge base with appropriate facts, predicates, entities, and rules.	10–12% – Knowledge base is well designed with minor inconsistencies or omissions.	7–9% – Basic knowledge base is developed but several facts/rules are missing or unclear.	0–6% – Knowledge base is incomplete, inconsistent, or inappropriate.
Propositional & First-Order Logic Representation	20%	17–20% – Correctly represents the domain using propositions, predicates, variables, constants, quantifiers, and logical connectives with excellent accuracy.	13–16% – Logic representation is mostly correct with minor errors.	9–12% – Provides basic representations but has several logical or syntactic errors.	0–8% – Logic representation is incorrect, incomplete, or missing.
Unification & Resolution	15%	13–15% – Correctly performs substitutions, identifies the MGU, and applies resolution systematically to derive valid conclusions.	10–12% – Applies unification and resolution correctly with minor errors.	7–9% – Demonstrates partial understanding but misses important steps or contains errors.	0–6% – Unable to correctly apply unification or resolution.

Forward Chaining & Backward Chaining	15%	13–15% – Correctly applies both forward and backward chaining with clear step-by-step reasoning and appropriate rule selection.	10–12% – Correctly applies both methods with minor errors or omissions.	7–9% – Demonstrates partial understanding of one or both chaining methods.	0–6% – Incorrectly applies or fails to demonstrate chaining techniques.
Inference & Reasoning Accuracy	10%	9–10% – Produces logically valid conclusions and clearly explains how facts and rules lead to the final inference.	7–8% – Conclusions are mostly correct with minor reasoning gaps.	5–6% – Some correct inferences but reasoning is incomplete or unclear.	0–4% – Conclusions are incorrect, unsupported, or missing.
Knowledge Representation Diagram / System Architecture	5%	5% – Provides a complete, accurate, well-labelled diagram showing knowledge base, inference engine, reasoning process, and output.	4% – Diagram is mostly correct with minor omissions.	3% – Diagram is partially complete or lacks clarity.	0–2% – Diagram is missing or incorrect.
Originality, Critical Thinking & Presentation	5%	5% – Demonstrates excellent independent thinking, appropriate real-world application, comparison of reasoning approaches, and clear presentation.	4% – Demonstrates good reasoning and clear presentation.	3% – Shows limited critical thinking and basic presentation.	0–2% – Shows little originality, weak reasoning, or poor presentation.